ls
1
grep
1
true
8
nl
9
grep
10
python3
11
python3
12
pytest
13
grep
14
22
pytest
23
echo
23
git
23
git
15
nl
16
grep
17
nl
18
grep
19
nl
20
python3
21
7
grep
7
6
2
nl
3
grep
3
nl
4
nl
5
grep
6
grep
localization
patch
validation
general

```
.../n        return'if old in s and insertion not in s:    s = s.replace(old, insertion, 1)    p.write_text(s)    print("patched")else:    print("no change needed")
```

```
.../import syssys.path.insert(0,'.')from types import simplenamespacefrom sphinx.ext.autodoc.typehints import record_typehintsclass app:    passapp = app()class config:    autodoc_typehints_format = 'short'    autodoc_type_aliases = {}app.config = config()app.env = simplenamespace(temp_data={})# define a class with __init__ return annotation -> noneclass square:    def __init__(self, width: int, height: int) -> none:        self.width = width        self.height = height# define a function with return annotationdef func(x: int) -> str:    return str(x)# call record_typehints for a class object (objtype 'class')record_typehints(app, 'class', 'mod.square', square, {}, none, none)print("after class:", app.env.temp_data)# reset temp_data and call for function (objtype 'function')app.env.temp_data = {}record_typehints(app, 'function', 'mod.func', func, {}, none, none)print("after function:", app.env.temp_data)
```

```
.../cls)            if bound and idx == 0 and param.name in ('self', 'cls'):                continue            if param.annotation is not param.empty:                annotation[param.name] = typing.stringify(param.annotation, mode)        if sig.return_annotation is not sig.empty:            annotation['return'] = typing.stringify(sig.return_annotation, mode)    except (typeerror, valueerror):        pass"""    s = s.replace(old_block_start, new_block, 1)    p.write_text(s)    print("patched")else:    print("pattern not found; no change made")
```